DEEP LEARNING APLICADO · CLASE PRESENCIAL

# Fine-Tuning

## de Modelos Pre-entrenados



*LoRA · Unsloth · Datasets · Herramientas*

Entrenamiento eficiente con recursos limitados

# ¿Qué es Fine-Tuning?

## La analogía

**Un médico general**
tiene conocimiento amplio del cuerpo humano.

**Un especialista cardiólogo**
aprendió primero medicina general y luego se especializó.

***Fine-Tuning = especialización.***
No empezamos desde cero — partimos del conocimiento general.

## La definición

- **Modelo pre-entrenado**
  Entrenado en miles de millones de tokens (GPT, LLaMA, Mistral…)
- **Dataset pequeño y específico**
  Cientos o miles de ejemplos de la tarea objetivo
- **Ajustar pesos**
  Continuar el entrenamiento con learning rate muy bajo
- **Resultado**
  Modelo que mantiene conocimiento general + dominio específico

# Pre-entrenamiento vs Fine-Tuning

## Pre-entrenamiento

**Datos:** Billones de tokens (internet completo)

**Costo:** Millones de USD · Semanas de GPU

**Objetivo:** Aprender lenguaje, razonamiento, hechos del mundo

**Quién:** OpenAI, Google, Meta, Mistral…

**Output:** Modelo base (foundation model)

## Fine-Tuning

**Datos:** Cientos a millones de ejemplos específicos

**Costo:** Horas a días · Una sola GPU (con LoRA)

**Objetivo:** Adaptar comportamiento a una tarea concreta

**Quién:** Empresas, investigadores, ¡vos!

**Output:** Modelo especializado

# LoRA — Low-Rank Adaptation

*La técnica que hace al fine-tuning accesible*

### El problema

LLaMA-3 8B tiene
**8.000 millones**
de parámetros.

Full fine-tuning =
entrenar todos → imposible
con 1 GPU.

### La solución LoRA

En vez de modificar W (el peso original),
agregas dos matrices pequeñas A y B:

$$W_{nuevo} = W_{original} + A \times B$$

- **W_original:** congelado (no se toca)
- **A, B:** matrices de rank bajo (r = 4, 8, 16…) → 1000x menos parámetros
- **Solo entrenamos A y B** — el modelo base permanece intacto

# Herramientas para Fine-Tuning

## Unsloth

⭐ Recomendada para clase

- ✓ 2x más rápido que HuggingFace estándar
- ✓ 60-80% menos VRAM con QLoRA
- ✓ Compatible con LLaMA, Mistral, Qwen, Phi…
- ✓ Corre en Google Colab gratis
- ✓ API idéntica a HuggingFace — fácil migración

## HuggingFace PEFT + TRL

Estándar de la industria

- ✓ PEFT: librería oficial para LoRA / QLoRA
- ✓ TRL: SFT, RLHF, DPO — todo en uno
- ✓ Documentación exhaustiva
- ✓ Más lento que Unsloth pero más flexible
- ✓ Compatible con todos los modelos de HF Hub

*También: Axolotl · LitGPT · LLaMA Factory · OpenLLM · Ollama (serving)*

# Los Datos: el factor más crítico

*"Garbage in, garbage out — con un modelo poderoso, garbage in, garbage out muy bien redactado"*

## Cantidad

- Instrucciones simples: 500–2.000 ejemplos
- Tarea compleja / dominio técnico: 10.000+
- Más no siempre es mejor — la calidad gana
- Regla práctica: empezá pequeño, medí,

## Calidad

- Consistencia: mismo estilo y tono
- Sin errores en las respuestas esperadas
- Diversidad: cubrir casos edge
- Un mal ejemplo pesa más que 10 buenos

## Formato

- Instruction-following: {input, output}
- Conversacional: {messages: [{role, content}]}
- Completion: texto plano para continuar
- Cada modelo tiene su chat_template

**Datasets públicos:** Alpaca · ShareGPT · OpenHermes · Dolly · LIMA · UltraChat — todos en huggingface.co/datasets

# El Proceso: paso a paso

**01** **Elegir modelo base**

LLaMA 3.1 8B, Mistral 7B, Qwen 2.5 7B
Más pequeño = más fácil de fine-tunear

**02** **Preparar el dataset**

Formato instruction-following
JSON con {instruction, input, output}

**03** **Configurar LoRA/QLoRA**

r=16, alpha=16, target_modules
Elegir capas a adaptar (q_proj, v_proj…)

**04** **Entrenar con Unsloth**

SFTTrainer con ~3 epochs
Monitorear train_loss / val_loss

**05** **Evaluar el modelo**

Probar manualmente + métricas
Benchmarks: MMLU, HellaSwag, etc.

**06** **Deploy / servir**

Exportar a GGUF → Ollama
O subir a HuggingFace Hub

# Código: Fine-Tuning con Unsloth

```
# 1. Instalar Unsloth
!pip install unsloth

# 2. Cargar modelo con 4-bit QLoRA
from unsloth import FastLanguageModel
model, tokenizer = FastLanguageModel.from_pretrained(
    model_name = "unsloth/llama-3.1-8b-instruct",
    max_seq_length = 2048,
    load_in_4bit = True,  # QLoRA
)

# 3. Agregar adaptadores LoRA
model = FastLanguageModel.get_peft_model(
    model, r=16, lora_alpha=16,
    target_modules=["q_proj","v_proj"],
)

# 4. Entrenar
trainer.train()  # ← ¡eso es todo!
```

**QLoRA —** Cuantiza el modelo base a 4-bit → cabe en 6-8 GB de VRAM (T4 de Colab)

**r=16 —** Rank de LoRA. A mayor r, más parámetros entrenados y más expresividad (pero más VRAM)

**SFTTrainer —** Supervised Fine-Tuning Trainer de HuggingFace TRL. Maneja batching, padding y chat templates automáticamente

**GGUF export —** Al terminar: model.save_pretrained_gguf() para usar con Ollama localmente

# ¿Son importantes los datos?

*Spoiler: son lo más importante.*

### ☑ Dataset de calidad

- Respuestas correctas y consistentes
- Diversidad de estilos y casos
- Sin sesgos sistemáticos
- Chat template correcto
- Revisado por humanos

→ ***Modelo útil, confiable, seguro***

### ✗ Dataset deficiente

- Respuestas incorrectas o parciales
- Repetición de patrones (overfitting)
- Sesgos amplificados
- Formato inconsistente
- Generado con IA sin revisión

→ ***Modelo que alucina más y con más convicción***

# Lo que vimos hoy

- Fine-Tuning = especializar un modelo base con datos específicos
- LoRA = entrenar solo matrices pequeñas → 1000x menos parámetros
- Unsloth + QLoRA = fine-tuning en la GPU gratuita de Colab
- Los datos son el factor #1 — calidad > cantidad
- Herramientas: Unsloth, HF PEFT, TRL, Axolotl, LLaMA Factory

**A continuación**

Video demo:
fine-tuning
con Unsloth
en Colab

https://www.youtube.com/watch?v=xMm66CnUVgw

https://www.youtube.com/watch?v=--l1HXN6gXc

https://www.youtube.com/watch?v=1Jr4QOFI0qw